WEEKLY MINI PROJECT 04 · TEAM 3

Bytes for Futures Assessment Platform

Team Work Split · Overview

5 work packages | One owner per file | Nobody waits

PROJECT	TEAM LEADER	TEAM MEMBERS
Bytes for Futures Assessment Platform	**Ali Mikdad**	**Maya Al Hamwi, Esmaeel Mahfoud, Majed Hmoud, Rami Al Shaar**

BACKEND	FRONTEND	LEADER
Esmaeel, Majed, Rami	**Ali, Maya**	**Ali: foundation, integration, review, demo**

A fair, timed, server-controlled assessment: the clock, the order of the tests and every score belong to the backend.

Every file has exactly one owner. You edit only your own files, you code against the fixed names, and the stubs from Phase 0 let you test before anyone else is done. Nawar Hasan is not part of this split.

What you already know from class: the same NestJS pattern from B4F Board (Module → Controller → Service, DTO validation, guards, Swagger, Logger, the global exception filter) and the Prisma work from Sessions 19–21 (`schema.prisma` , generated migrations, `$transaction` , `{ increment: 1 }` , conditional updates, the service-level cutover). Nobody starts from zero.

Full reference: this package is a summary of `docs/TASKS.md` . Every name, endpoint, shape and message is fixed in `docs/API.md` and `docs/PROJECT-STRUCTURE.md` . If this package and the docs ever disagree, the docs win; tell Ali.

1 Who Does What

Member	Builds	Owns	Hours
Ali Mikdad · Leader · Frontend	Frontend foundation, Auth screens, Instructor screens, Exam-guard toolkit, Review & Integration	`client/` skeleton, `pages/Auth/` , `pages/instructor/` , `exam-guard/` , `docs/` , `.github/`	~18 h
Maya Al Hamwi · Frontend	Student screens: My Tests, Test Intro, MCQ test (with autosave), Coding test (editor, Run, Submit), My Results + exam-guard wiring	`pages/student/` , student contexts, `student.api.ts` , `CodeEditor` , `Countdown`	~17.5 h
Esmaeel Mahfoud · Backend	Backend foundation, Prisma schema, Seed, Auth & Roles, Exam integrity (violations, termination, session lock), Attempts list & Results, Endpoint testing	`main.ts` , `common/` , `prisma/` , `auth/` , `integrity/` , `results/` , `schema.prisma` , `api.http`	~17.5 h

Member	Builds	Owns	Hours
Majed Hmoud · Backend	Test content (publish, questions, problems, MCQ seed, shuffled order), Code Execution Service (mock + Judge0, harness, readable errors)	`tests/`, `execution/`, `seed-data/mcq-questions.ts`, `seed-data/coding-problems.ts`	~17.5 h
Rami Al Shaar · Backend	Attempt state & timing, Scoring, Student tests, Start (pledge, order, session), MCQ autosave & submit, Coding Run / Submit / Finish	`attempt-state/`, `student/`	~18 h

The hours are targets, not promises. If one member finishes early, they help **by testing and reviewing**, not by editing someone else's files.

Working pairs

Feature	Backend	Frontend
Sign in / Sign up	Esmaeel	Ali
Test management + editor	Majed	Ali
Instructor attempts	Esmaeel	Ali
My Tests + Test Intro + start	Rami	Maya
MCQ test	Rami	Maya
Coding test (Run, Submit, Finish)	Rami + Majed (execution)	Maya
My Results	Esmaeel	Maya
Exam integrity (full screen, tab switch, copy/paste, termination, session)	Esmaeel (+ Rami's scoring)	Ali (toolkit) + Maya (wiring)

Reviews

- Ali approves the Pull Requests of everyone else.
- Maya approves Ali's frontend Pull Requests. Esmaeel approves Ali's PRs that touch `docs/` only.
- Backend PRs also get a quick look from one other backend member before Ali approves.

2 Work Without Waiting

PHASE 0 · DAY 1 MORNING

Stubs everywhere

Ali and Esmaeel push both skeletons. Every endpoint answers with the example JSON from `API.md`; every route shows a placeholder.

PHASE 1 · DAYS 1–2

Build alone

Everyone works in their own files, against the fixed names. Replace stubs only inside your own module.

PHASE 2 · END OF DAY 2

Real integration

Every stub replaced, every context on the real endpoints, every message checked against `API.md`.

Your work depends on `docs/API.md` and `docs/PROJECT-STRUCTURE.md`, not on someone's unfinished code. If a dependency is not merged yet, code against the fixed name and test with the stub.

3 Frozen API Contract

Base URL `http://localhost:3000/api` · Swagger `http://localhost:3000/api/docs` · every protected request sends `Authorization: Bearer <accessToken>` · every route under `/student/attempts/:attemptId` also sends `X-Attempt-Session` (`API.md` 19.4).

Method	Endpoint	Purpose	Role	Owner
POST	`/auth/register`	Student sign-up	public	Esmaeel
POST	`/auth/login`	Sign in (both roles)	public	Esmaeel
GET	`/auth/me`	Current user	any	Esmaeel
GET	`/instructor/tests`	The three tests with counts	INSTRUCTOR	Majed
GET	`/instructor/tests/:testId`	One test with its questions or problems	INSTRUCTOR	Majed
PATCH	`/instructor/tests/:testId/publish`	Publish a test	INSTRUCTOR	Majed
PATCH	`/instructor/tests/:testId/unpublish`	Unpublish a test	INSTRUCTOR	Majed
POST	`/instructor/tests/:testId/questions`	Add an MCQ question	INSTRUCTOR	Majed
PUT	`/instructor/questions/:questionId`	Replace an MCQ question	INSTRUCTOR	Majed
DELETE	`/instructor/questions/:questionId`	Delete an MCQ question	INSTRUCTOR	Majed
POST	`/instructor/tests/:testId/problems`	Add a coding problem	INSTRUCTOR	Majed
PUT	`/instructor/problems/:problemId`	Replace a coding problem	INSTRUCTOR	Majed
DELETE	`/instructor/problems/:problemId`	Delete a coding problem	INSTRUCTOR	Majed
GET	`/instructor/attempts?testId=&status=&search=`	All attempts, filtered	INSTRUCTOR	Esmaeel
GET	`/instructor/attempts/:attemptId`	One attempt in detail	INSTRUCTOR	Esmaeel
GET	`/student/tests`	Published tests with their state	STUDENT	Rami
GET	`/student/tests/:testId`	One published test with its state	STUDENT	Rami
POST	`/student/tests/:testId/attempts`	Start (or resume) an attempt	STUDENT	Rami

Method	Endpoint	Purpose	Role	Owner
GET	`/student/attempts/:attemptId`	The attempt screen data	STUDENT	Rami
POST	`/student/attempts/:attemptId/mcq-submit`	Submit MCQ answers	STUDENT	Rami
POST	`/student/attempts/:attemptId/problems/:problemId/run`	Run code on the sample case	STUDENT	Rami (+ Majed's execution service)
POST	`/student/attempts/:attemptId/problems/:problemId/submit`	Submit code for hidden tests	STUDENT	Rami (+ Majed's execution service)
POST	`/student/attempts/:attemptId/finish`	Finish the coding test	STUDENT	Rami
PUT	`/student/attempts/:attemptId/mcq-answers/:questionId`	Autosave one MCQ answer	STUDENT	Rami
POST	`/student/attempts/:attemptId/violations`	Report an exam-rule violation	STUDENT	Esmaeel
GET	`/student/results`	The student's own results	STUDENT	Esmaeel

Every error has the shape `{ "statusCode": 409, "message": "Human-readable reason." }` with **one** string. The messages in `API.md` are fixed; the frontend shows them as they are.

4 Frozen Data Model

PostgreSQL through Prisma 6.7.0. `schema.prisma` has one owner (Esmaeel) and is complete after Phase 0, including the integrity fields. Full schema: `PROJECT-STRUCTURE.md` 7.2.

Model	Fields	Unique / index
`User`	id, fullName, email, passwordHash, role (`INSTRUCTOR` | `STUDENT`), createdAt	`email`
`Test`	id, title, type (`MCQ` | `CODING`), order, durationMinutes, status (`DRAFT` | `PUBLISHED`)	`order`
`Question`	id, testId, text, position	—
`Option`	id, questionId, text, isCorrect, position	—
`Problem`	id, testId, title, description, functionName, points, position	—
`TestCase`	id, problemId, visibility (`SAMPLE` | `HIDDEN`), args (Json), expected (Json), position	—
`Attempt`	id, userId, testId, status (`IN_PROGRESS` | `SUBMITTED` | `EXPIRED` | `TERMINATED`), startedAt, deadline, submittedAt, terminatedAt, terminationReason, score, sessionKey, questionOrder, optionOrder	`(userId, testId)` unique; index `(testId, status)`

Model	Fields	Unique / index
`McqAnswer`	id, attemptId, questionId, optionId?, isCorrect	`(attemptId, questionId)`
`ProblemResult`	id, attemptId, problemId, runCount, code, passedCount, totalCount, points, submittedAt	`(attemptId, problemId)`
`AttemptViolation`	id, attemptId, type, counted, occurredAt	index `(attemptId, occurredAt)`

- `score` is `null` for `IN_PROGRESS` and `EXPIRED`. `TERMINATED` has a real score.
- Every schema change is a new generated migration. Never `prisma migrate reset` on shared data.

5 Interfaces Between Members

These names are fixed from Phase 0. Code against them even before the owner's PR is merged.

Used by	Provided by	Name
Majed, Esmaeel, Rami	Rami	`AttemptStateService.expireOverdue()`, `hasAttempts(testId)`, `hasActiveAttempts(testId)`, `assertAcceptingRequests(attempt)`
Rami	Majed	`CodeExecutionService.run(problem, code)`, `submit(problem, code)`, `getMode()`
Rami	Majed	`buildAttemptOrder(questions)`, `applyQuestionOrder()`, `applyOptionOrder()`
Esmaeel	Rami	`ScoringService.scoreMcqFromSaved(tx, attemptId)`, `scoreCoding(tx, attemptId)`
Rami	Esmaeel	`IntegrityService.issueSessionKey(tx, attemptId, isResume)`, `getIntegrity(attemptId)`, `AttemptSessionGuard`
Maya	Ali	`useFullscreen()`, `useExamGuard()`, `attachEditorPasteGuard()`, `FullscreenGate`, `ViolationWarningDialog`
Everyone (backend)	Esmaeel	`ClockService.now()`, `round1()`, constants, `JwtAuthGuard`, `RolesGuard`, `@Roles()`, `@CurrentUser()`, id param DTOs
Maya	Ali	`components/common/*` props, `http.ts` `request<T>()`, `ApiError`, `useAuth()`, `formatScore()`
Ali, Maya	Backend	Every shape and message in `API.md`

6 Exam Integrity at a Glance

> **Instructor-approved addition.** The specification (section 17) lists anti-cheat as out of scope. Keep the written approval linked in `PROJECT-STRUCTURE.md`, section 3.

Group	Violation types	Limit	What happens
`FOCUS`	`FULLSCREEN_EXIT`, `TAB_HIDDEN`	2	1st → warning. 2nd → attempt `TERMINATED`.
`CLIPBOARD`	`COPY`, `CUT`, `PASTE_EXTERNAL`, `DROP_EXTERNAL`	3	1st and 2nd → warning. 3rd → attempt `TERMINATED`.
`SESSION`	`NEW_SESSION`	—	Logged for the instructor. Never counted. Sent only by the server.

- The browser only **reports**. The backend counts, decides the warning or the termination, and stores everything.

- A terminated attempt keeps the score of what was saved before the termination (MCQ: autosaved answers; coding: submitted problems), always shown with `Terminated — exam rules violated`. It unlocks the next test.
- Also: shuffled question and option order per student, one active session per attempt, an exam-rules pledge before Start, full screen required (computer only).

7 Decisions That Need the Instructor's Approval

Decision	Why it needs approval	If refused
Real execution through Judge0	Specification section 16 (CHALLENGE)	`EXECUTION_MODE=mock`. Nothing else changes.
Monaco Editor instead of a text area	Specification section 15 asks editor improvements "with plain React"	`CodeEditor` falls back to a `<textarea>`. Same props.
Exam integrity: full screen, tab switching, copy/paste, termination	Specification section 17 lists anti-cheat as out of scope	**Approved by the instructor.**

Mock mode is CORE and stays available in every case (R-23). Everything must work in Syria **without a VPN**: no CDN, no Google Fonts, the browser talks only to our backend (`PROJECT-STRUCTURE.md` section 6).

8 Timeline

When	Goal
Phase 0 · Day 1 morning	Ali: repository + frontend skeleton. Esmaeel: backend skeleton + schema + migration + stubs. Majed: execution spike. Maya: student screen sketches. Rami: attempt state table.
Phase 1 · rest of Day 1 and Day 2	First the base PRs (attempt state + scoring, auth + guards, execution interface + mock + order util, shared UI, countdown + session helper), then everyone builds alone.
Phase 2 · end of Day 2	Real integration: every stub replaced; Esmaeel runs `api.http`; Ali runs the full test.
Phase 3 · Day 3 morning	Fix and strengthen: every rule and error case, real execution errors, desktop and phone, the no-VPN check, the stop-the-backend test.
Phase 4 · Day 3 afternoon	Polish, README, seed data, start rehearsing.
Phase 5 · Day 4 (or demo day)	Code freeze, final full test on a clean clone, two rehearsals.

Merge order that avoids blocking

```
Phase 0 foundation
 ↓
attempt-state + auth/guards + execution interface/mock + shared UI + countdown   (the "base" PRs)
 ↓
content (Majed), student tests/start/MCQ (Rami), attempts/results (Esmaeel)
 ↓
integrity module (Esmaeel) + exam-guard toolkit (Ali)
 ↓
harness → comparer → error normalizer → Judge0 provider (Majed)
 ↓
coding run → submit → finish (Rami)
 ↓
pages replace the stub assumptions (Ali, Maya)
```

Daily sync (15 minutes): what did I finish? what will I do next? what blocks me?

9 Git Rules

- **Never push directly to `main`.** Run `git branch --show-current` before every commit.
- Branches: `be/<module>-<description>` or `fe/<page>-<description>`. One task = one branch.
- Commits: `type(scope): message` (`feat`, `fix`, `refactor`, `style`, `docs`, `test`, `chore`).
- Pull the latest `main` before you start and before you open a PR (`git fetch origin` + `git merge origin/main`).
- Keep PRs small (ideally under 300 changed lines). One or two PRs a day.
- Change only files you own. A conflict in a file you do not own? Stop and tell Ali.
- Never commit `node_modules`, `dist`, `.env`, keys, or tokens.
- Never run `npm install <package>` yourself. Ask Ali (`client/`) or Esmaeel (`api/`).
- Never edit `schema.prisma` yourself. Ask Esmaeel.
- Never use `git push --force` on a shared branch. Format with Prettier before you commit.
- Blocked by someone? Say it in the team chat right away.
- Code freeze two hours before the demo: only bug-fix PRs approved by Ali.